

Attention Drift: What Autoregressive Speculative Decoding Models Learn *

Doğaç Eldenk
Northwestern University

Payal Mohapatra
Northwestern University

Yigitcan Comlek
GE Aerospace

Kaan Oktay
fal

Hongyang Zhang
University of Waterloo

Stephen Xia
Northwestern University

Abstract

Speculative decoding accelerates LLM inference by drafting future tokens with a small model. However, drafter models degrade sharply under template perturbation and long-context inputs. We identify a previously-unreported phenomenon we call **attention drift**: as the drafter generates successive tokens within a speculation chain, attention progressively moves from the prompt onto its own recently-generated tokens. This phenomenon is observed across both *EAGLE3* drafters and *MTP heads*, suggesting drift is a property of drafter designs. We trace this to the unnormalized residual path between chain steps: the drafter’s hidden state magnitude grows monotonically with chain depth, which exhibits dynamics consistent with additional pre-norm transformer layers stacked on the target rather than as a standalone autoregressive predictor. In order to limit the growth, we propose two architectural changes: Post-norm on the drafter hidden states and per-hidden-state RMSNorm after capturing target hidden states. Our interventions improve acceptance length over the current leading model, pre-norm *EAGLE3*, by up to $2\times$ under template perturbation, $1.18\times$ on long-context tasks, and $1.10\times$ on seven standard benchmarks spanning multi-turn chat, math, and coding. Our changes also allow shorter train-time-test depths to generalize over longer drafting sequences.

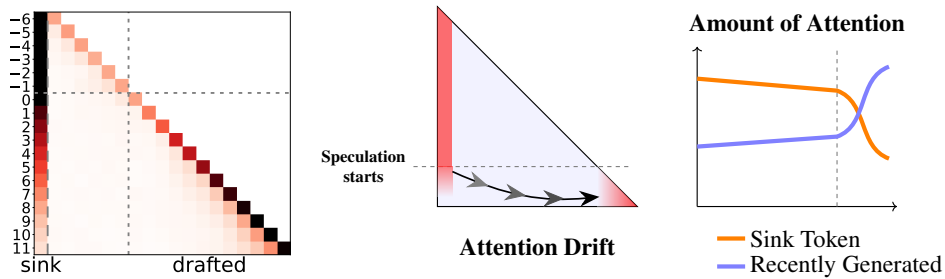


Figure 1: **Attention drift.** During speculation, the drafter’s attention moves from the prompt’s sink token onto its own recently-generated tokens. *Left:* Emergence of the attention sink demonstrated on drafter’s attention heatmap, rows = query, columns = key; darker = higher attention. *Center:* Graphical visualization of attention drift on a drafter. *Right:* Attention per token position on x axis, with speculated tokens to the right of the dashed line.

*Code: <https://github.com/Dogacel/Attention-Drift>

1 Introduction

Speculative decoding [1, 2] is a lossless acceleration technique for large language model (LLM) inference in which a lightweight *drafter* predicts future tokens that are later verified by a larger *target* or *verifier* model. In practice, speculative decoding systems must operate under a wide range of deployment conditions, including different inference engines, context lengths, system prompts, and chat templates. Robustness under varying scenarios is critical to maintaining high acceptance rates and consistent inference speedups. However, recent works have shown that drafters often degrade substantially under challenging deployment scenarios, such as template perturbations and long-context inputs [3, 4]. As a result, practitioners frequently retrain or specialize drafters for specific inference engines, prompts, or templates to maximize performance. Despite speculative decoding models being relatively cheap to train, this sensitivity points to an underlying issue. Our main contributions in this work are as follows:

(i) We identify a previously unreported phenomenon that helps explain this fragility, which we call **attention drift** (Figure 1). As a drafter generates more tokens during speculation, its attention progressively shifts away from the sink toward recently generated tokens. We observe this phenomenon consistently across EAGLE-3 drafters [5] and Multi-Token Prediction (MTP) heads [6], suggesting that attention drift reflects a broader property of auto-regressive drafter designs.

(ii) To understand this behavior, we analyze the hidden-state dynamics of speculative drafters and find that the unnormalized residual connection between speculation steps causes hidden-state magnitudes to grow monotonically with chain depth, resembling additional transformer layers stacked on top of the verifier. The drafter implicitly learns a depth-dependent refinement process instead of a stable autoregressive token predictor, making it sensitive to template changes and long contexts (Section 4.2).

(iii) Motivated by this observation, we introduce a simple architectural modification based on *post-normalization* (Figure 2) combined with a normalization before hidden-state fusion, that prevents hidden-state magnitude growth and stabilizes the drafting process. These changes substantially reduce attention drift and improve acceptance length over the current state-of-the-art pre-norm EAGLE3 architecture by up to $2\times$ under template perturbation, $1.18\times$ on long-context tasks, and $1.10\times$ across seven standard benchmarks spanning multi-turn chat, reasoning, coding, and mathematics. We further show that post-norm drafters generalize to inference depths beyond those seen during training, enabling shorter train-time-test depths and reducing training cost.

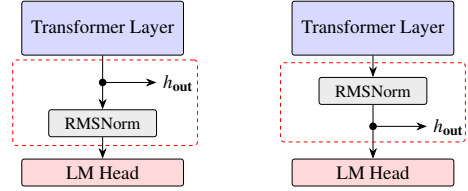


Figure 2: Overview of Pre-norm (Left) and proposed Post-norm (Right) architecture.

2 Preliminaries

Speculative Decoding alternates between two stages (Figure 3). In the drafting phase, the drafter auto-regressively generates a sequence of k candidate future tokens, where k is the *speculation depth*. In the verification phase, the target model evaluates the drafted sequence in a single forward pass and applies rejection sampling against its own token distribution; the longest valid prefix is accepted. A key efficiency metric is the *acceptance length* (τ), defined as the average number of drafted tokens accepted per verification round. Higher acceptance lengths generally translate directly into larger end-to-end inference speedups for a fixed drafter overhead.

EAGLE [7] and its successors EAGLE-2/EAGLE-3 [8, 5] feed the target’s fused internal hidden states through a fully-connected projection (FC in Figure 13) into a single pre-norm transformer decoder layer [9] that serves as the drafter (Figure 2 Left). The drafter is trained with a token-level cross-entropy loss against the frozen target’s predicted distribution, with a fixed number of speculation steps (the train-time-test depth, TTT).

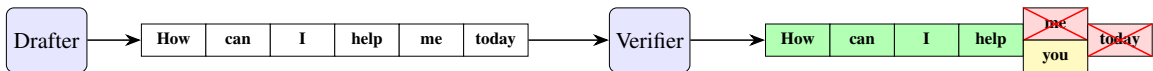


Figure 3: Verification phase: green tokens are accepted, yellow is resampled and red ones are rejected.

EAGLE-3, our focus, is the dominant auto-regressive drafter design in production engines such as vLLM [10] and SGLang [11].

Attention Sinks. An attention sink [12] is a token (typically near the start of a sequence) that absorbs disproportionately large attention during inference. Sinks are widely observed in modern LLMs and are believed to act as stable anchors that stabilize attention, particularly under long-context.

3 Attention Drift

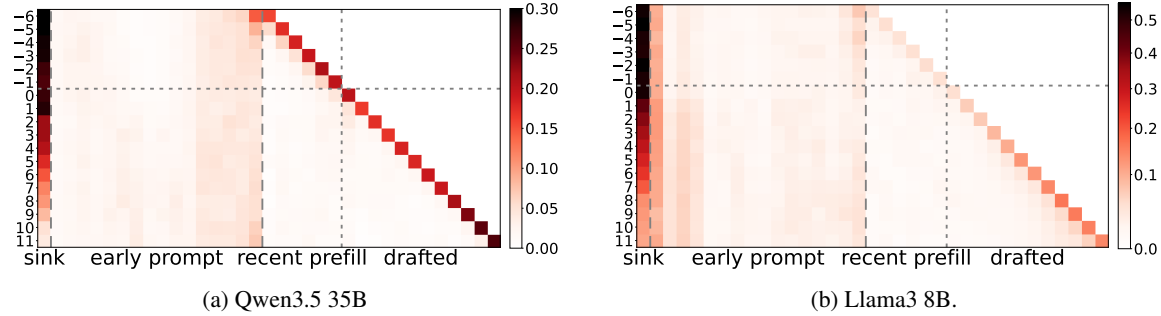


Figure 4: Attention heatmaps for visualizing attention drift on EAGLE-3 drafters. Aggregated over 200+ samples from varied prompts and sequence positions.

Looking at speculative decoding models’ attention, a pattern emerges: **whenever the target model develops a sink, the drafter develops a sink at exactly the same place.** This observation is consistent across **MTP heads** and **EAGLE-3** drafters. Upon close inspection of three model families with different attention patterns, we observed Llama’s sink is on the first token, Qwen’s on the second, and GPT-oss has no sink. In every case, the drafter matched its target.

The drafter attention resembles the verifier attention as the drafter consumes verifier hidden states. However, once drafting begins, the attention distribution progressively changes as the drafter transitions from consuming verifier hidden states to consuming its own generated hidden states. During this process, attention increasingly shifts away from the sink and toward recently generated tokens.

We refer to this phenomenon as **attention drift**, visualized in Figures 4 and 5 across multiple EAGLE-3 drafters. In models with strong sinks, the sink progressively weakens during drafting while attention mass concentrates on recently drafted tokens. The drafter therefore operates in two modes: attending to the verifier’s hidden states at pre-fill and to its own hidden states during generation, with a gradual learned transition between them. We hypothesize that this transition makes the drafter **more fragile to out-of-distribution inputs**, most notably in long context and at deeper speculation steps.

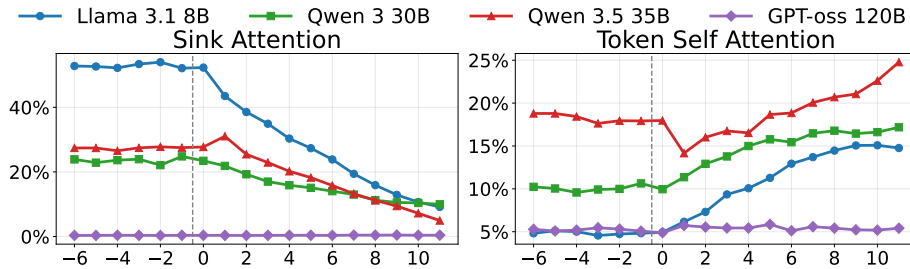


Figure 5: Percentage of attention concentrated on Sink token (Left) and Latest Token (Right) on EAGLE-3 drafters.

Here, we examine MTP Heads [6] as a cross-architecture check on whether attention drift is specific to EAGLE-3 or a more general property of drafters that consume their own previous outputs. MTP Heads are auxiliary prediction heads jointly trained with the target model during pretraining, sharing the target’s LM head and predicting multiple positions ahead. This differs from EAGLE-3, which is trained post-hoc as a separate drafter on top of a frozen target with its own LM head.

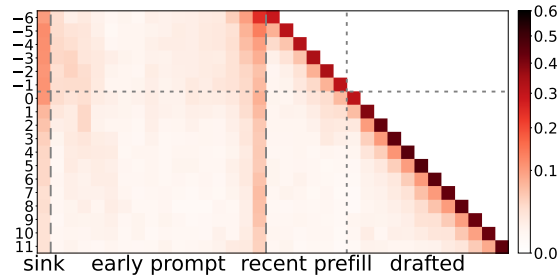


Figure 6: Attention Drift on Qwen3.5 9B MTP head

We inspect the MTP head of Qwen3.5 9B (Section 3), a single transformer layer that follows the target model’s attention architecture and reuses the same weights across consecutive speculation steps. During MTP speculation, we observe similar trends (Figure 7): attention to the sink token decreases substantially, while attention to the most recently drafted token increases.

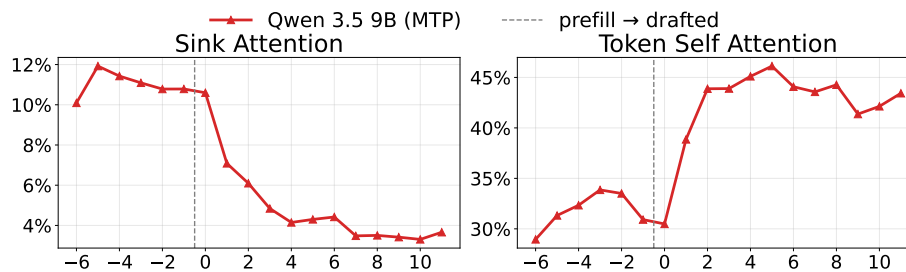


Figure 7: Sink and drafted token self attention on Qwen3.5 9B MTP heads (MT-Bench, 80 prompts).

What if verifier doesn’t have an attention sink? Some recent models, such as Qwen3-Next and GPT-oss, are designed to suppress attention sinks. Qwen3-Next uses gated attention, applying a per-head sigmoid gate to the SDPA output so that heads can multiplicatively suppress their contribution. GPT-oss instead introduces a per-head learnable bias logit in the softmax denominator, giving each head an explicit "attend to nothing" option that absorbs excess attention mass. Inspecting a GPT-oss drafter (Figure 8), we observe no visible sink on the first token, but a weak yet consistent attention concentration on recurring template tokens. This suggests that sink-like behavior can arise not only from architectural inductive bias but also from repeated special tokens or template markers, especially in the absence of a dedicated architectural sink mechanism in the verifier.

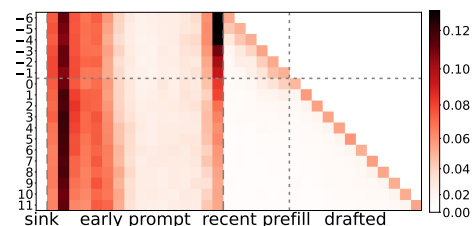


Figure 8: Attention Drift on GPT-oss 120B.

4 What Causes Attention Drift?

To understand why attention drift occurs and what it reveals about the underlying model, we trained drafters from three different verifier model families. We primarily evaluate on MT-Bench [13], eight categories of multi-turn instructions allows us to evaluate across diverse prompts. All architectural changes explored in this section required re-training of the drafter (details in Appendix C).

4.1 Hidden State Magnitudes

Attention drift occurs as we move away from the target’s hidden states towards consuming the drafter’s own hidden states during generation. Therefore we first inspect the hidden states of the verifier and drafter. In Table 1, the magnitudes are compared between the verifier’s hidden states, verifier’s fused hidden state and the drafter’s own hidden states at different speculation steps, using root mean square (RMS), $\|x\|_2/\sqrt{d}$. We notice several patterns described next.

Table 1: Hidden state magnitudes (RMS) across model families, averaged over 80 MT-Bench prompts and $k=8$ speculation rounds. Hidden states from the verifier’s low, middle, and high layers (h_{low} , h_{mid} , h_{high}); the fused representation of the verifier’s hidden states h_{FC} ; and the drafter’s output hidden states at speculation steps 1, 2, 3, and 8 (h_1^* through h_8^*)

Target Model	Verifier hidden states			h_{FC}	Drafter output h_t^*			
	h_{low}	h_{mid}	h_{high}		$k=1$	$k=2$	$k=3$	$k=8$
Llama 3.1 8B	0.56	0.58	0.78	12.46	3.92	4.87	5.86	14.02
Qwen 3 30B	0.33	2.17	2.71	2.56	1.00	1.13	1.25	1.67
Qwen 3.5 35B	0.03	0.12	0.21	0.89	3.47	4.03	4.42	5.92
GPT-oss 20B	3	52	324	96	87	89	95	138
GPT-oss 120B	3	163	1455	583	497	511	537	647

Observation 1: Hidden-state magnitudes mismatch. The hidden states of the verifier, target, and the fully connected layer have substantially different magnitudes. This is attributed to the positioning of the RMSNorm in the drafter and a norm component being shared between (h_{FC}) and the drafter (h^*) shown in Figure 13. Despite the hidden state magnitudes being vastly different between the target and drafter, we did not see a correlation between the magnitude of hidden states and acceptance rates for models, hinting us that the model learns to compensate for it.

Observation 2: The verifier fused representation, h_{FC} , is imbalanced. All three target families we explored use the pre-norm architecture: RMSNorm is applied inside each transformer block (before attention and MLP), but never to the residual stream itself. As such, each layer’s magnitudes accumulate monotonically through depth (prenorm dilution [14]), and $\|h_1\| < \|h_2\| < \|h_3\|$ holds for every row of Table 1. The target’s final pre-LM-head RMSNorm normally absorbs this growth when producing logits, but EAGLE-3 uses verifier’s internal states captured before this norm (h_{low} , h_{mid} , h_{high}). Because of this h_{high} dominates the h_{FC} signal with its larger magnitude. Thus in future sections **we add RMSNorm layers before each target hidden stream** used as input to generate h_{FC} to prevent this imbalance and generate a more stable verifier representation.

Observation 3: Magnitude growth across speculation depth. The drafter-generated hidden states grow monotonically with speculation depth. Across all model families, h_k^* accumulates magnitude away from the distribution the drafter was trained on at step 0. This means that the drafter does not operate on a depth-invariant hidden-state distribution. Instead, each speculation step changes the scale of the representation consumed by the next step.

How is drift and magnitude related? This scaling magnitude across speculation depth is important in computing attention. Attention depends on computing query-key similarities. As the drafter generates deeper into the speculation chain, its current hidden state increasingly resembles recently generated drafter states rather than verifier-conditioned prompt states. The empirical consequence, quantified in the next section, is a *redistribution* of attention mass away from the original input and onto the drafter’s own predictions.

4.2 Norm Position Changes: Layer-stacking vs Autoregression

The monotonic growth of the drafter hidden states suggests a simple interpretation of what a drafter *learns to be*. In a standard pre-norm drafter, the drafter’s chain hidden states $h_1^*, h_2^*, \dots, h_k^*$ accumulate through an unnormalized residual path, causing the hidden states to monotonically grow with speculation depth. This makes the drafter behave less like an independent autoregressive model and more like a sequence of additional transformer layers stacked on top of the target. At speculation step k , the drafter effectively approximates what would happen if the target had $N, N+1, \dots, N+k$ layers, rather than acting as a standalone depth-invariant

model that autoregressively predicts the next token. The model takes the target’s representations and *keeps refining it* with another layer of attention+MLP compute, one per speculation step (Figure 9).

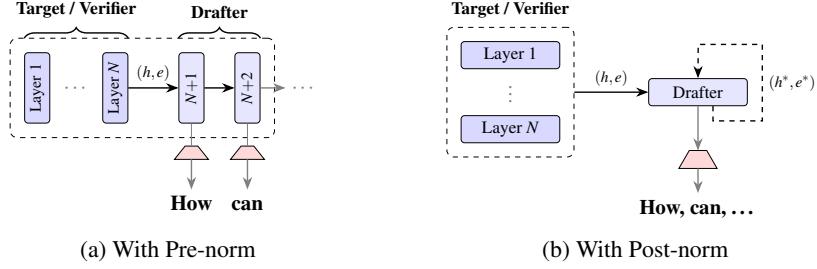


Figure 9: Two views of the EAGLE3 drafter at chain depth > 1 . **With the standard pre-norm structure, the model acts more like additional layers stacked onto the target model, whereas post-norm acts more like an independent auto-regressive drafter that accepts its own output.**

To test this interpretation, we compare standard pre-norm and the modified post-norm drafters trained with different train-time-test depths. Train-time-test depth denotes the maximum speculation depth used during training. If a drafter learns a depth-invariant autoregressive rule, then training with a small depth should still generalize reasonably to longer speculation chains at inference. In contrast, if the drafter learns depth-specific transformations resembling layers $N, N+1, \dots$, then it should fail beyond the depths observed during training. Figure 10 supports the latter interpretation for pre-norm drafters. A pre-norm drafter trained with train-time-test depth 2 performs well at the first few drafting steps but collapses beyond its training horizon. Its hidden-state magnitude grows rapidly, and its conditional acceptance probability drops sharply at deeper speculation steps.

By contrast, the post-norm drafter trained with the same TTT depth remains stable beyond the training horizon. Applying post-normalization to the drafter hidden state between speculation steps prevents residual-scale accumulation, making the drafter less able to encode depth through magnitude growth. Rather than continuing the verifier as a depth-dependent stack of additional pre-norm transformer layers, the post-norm drafter is regularized toward a stable autoregressive prediction function.

Drafter / TTT	h_1^*	h_8^*	$k=1$	$k=16$	Accept.
Pre-norm / 8	4.15	13.10	0.91	0.50	7.12
Pre-norm / 2	9.76	44.52	0.93	0.00	2.64
Post-norm / 2	1.22	1.30	0.93	0.53	5.05

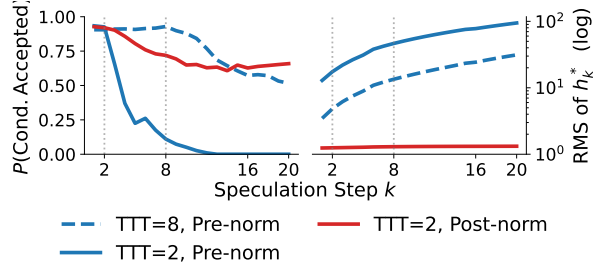


Figure 10: Pre-norm vs. Post-norm at various TTT (max training depth). Magnitude and per-step acceptance on coding (HumanEval) and math (GSM8K), $k=20$ steps.

4.3 Fixing the Attention Sink

The observation above suggests that hidden-state magnitude accumulation is a major contributor to attention drift. However, attention drift could also be interpreted as a consequence of attention-sink collapse: if the sink weakens during drafting, attention mass must move elsewhere. In this section, we test whether eliminating the sink is sufficient to solve drift as well as the magnitude growth.

To test this, we compare several architectural variants on the same target: a standard pre-norm drafter, a gated-attention drafter, a post-norm drafter, and combining gated-attention and post-norm. The gated-attention variant introduces a learned element-wise gate inside the attention layer (Appendix A), which strongly suppresses reliance on a fixed sink token. Table 2 summarizes the results.

Table 2: Gated attention vs. normalization: chain magnitude of hidden states h_k^* , entropy H , sink and recent token attention across EAGLE3 drafters for Llama-3.1 8B on MT-Bench (80 prompts over 8 categories and all drafting rounds).

Drafter	$h_{1 \rightarrow 8}^*$	$H_{1 \rightarrow 8}$	sink $_{1 \rightarrow 8}$	recent $_{1 \rightarrow 8}$
Pre-norm	3.92 $\rightarrow$ 14.02	2.04 $\rightarrow$ 2.73	0.46 $\rightarrow$ 0.08	0.14 $\rightarrow$ 0.31
Gated-Attn	8.41 $\rightarrow$ 39.07	2.39 $\rightarrow$ 2.13	0.03 $\rightarrow$ 0.02	0.29 $\rightarrow$ 0.32
Post-norm	1.21 $\rightarrow$ 1.20	2.24 $\rightarrow$ 2.29	0.11 $\rightarrow$ 0.10	0.50 $\rightarrow$ 0.53
Gated + post-norm	0.87 $\rightarrow$ 0.88	0.63 $\rightarrow$ 0.64	0.00 $\rightarrow$ 0.00	0.50 $\rightarrow$ 0.51

Magnitude and attention are independent failure modes. Gated attention nearly eliminates sink attention and removes the visible drift pattern (≤ 0.03 throughout), but its hidden state magnitude grows $5.0\times$ during drafting and starts from a larger $h_1^* \approx 8.4$. On the other hand, post-norm pins the hidden-state magnitude across speculation steps and stabilizes the attention pattern over the drafting chain. These results indicate that (i) fixing attention sink eliminates the drift, (ii) attention drift and magnitude accumulation are independent failure modes and that **solving attention drift does not necessarily eliminate the magnitude issue.**

Role of Entropy. We also examine whether attention entropy (H) explains drafter failures. Each model sits at a different base entropy level, and per-token entropy rises slightly as the speculation depth k grows, the distribution becomes differently-*shaped*, not peakier. However, no consistent relationship between entropy and acceptance length emerges across our pre-norm, post-norm, and gated-attention variants, indicating that drafters learn to compensate for entropy shifts and that entropy alone is not a useful predictor of drafter quality across architectures.

Combining the Gated Attention with Post-norm creates a new pathology. The Post-norm + Gated variant combines both modifications, and it does what one would hope along each of the two individual axes: chain magnitude is flat, the sink is eliminated, and the recent token attention is stable at 0.50. But entropy collapses to $H \approx 0.62$ already at pre-fill, about a third of the other drafters’ values, corresponding to an effective attention support of roughly two positions ($e^{0.62} \approx 1.85$). Applying both changes appears to over-regularize the drafter: the attention distribution collapses onto roughly two tokens.

4.4 Noise and Error Accumulation

The drafter consumes its own predictions as inputs to subsequent speculation steps, and these self-generated hidden states are noisier than the clean verifier states seen at $t=1$. The α -noise sweep therefore tests how gracefully each architecture absorbs imprecise hidden-state inputs, determining how quickly errors compound with depth. **Post-norm tolerates an order of magnitude more perturbation than pre-norm** on the hidden pathway (58% vs. 5% at $\alpha=0.5$), offering a mechanistic explanation for its better deep-chain behavior: it accumulates less error per speculation step. We further hypothesize that this tolerance may translate to robustness under other small hidden-state perturbations, such as those induced by verifier quantization or mild distribution shift.

Table 3: Drafter pathway reliance under noise injection. Each cell reports acceptance length as a percentage of the model’s no-perturbation baseline (parenthesized in the model column). Noise is scaled per-tensor RMS: $x \leftarrow x + \alpha \cdot \text{rms}(x) \cdot \epsilon$.

Model	Target	$\alpha=0.1$	$\alpha=0.25$	$\alpha=0.5$	$\alpha=1.0$	$\alpha=2.0$	$\alpha=\infty$
Pre-norm (3.06)	Hidden States	82%	28%	5%	0%	0%	0%
	Embeddings	99%	93%	86%	64%	27%	9%
Post-norm (3.16)	Hidden States	99%	86%	58%	22%	5%	2%
	Embeddings	98%	100%	93%	75%	38%	9%

Does artificially shrinking the magnitude fix attention drift? We observed that fixing the drift does not necessarily fix the magnitude accumulation. However we are not sure whether fixing the magnitude alone

would fix attention drift. We have created an inference-time controlled experiment, the drafter’s hidden states (h_{out}) were normalized to match the fc ’s magnitude during inference. Table 4 shows that normalizing the magnitude without matched training hurts the accuracy in both cases, pre-norm being effected even more than post-norm. However we observe one interesting thing, the attention drift is significantly lessened on pre-norm architecture, while still happening weakly. This shows that magnitude accumulation is one of the contributors to the attention drift but not the only one. Note that the experiment isolates the magnitude-attention link at inference but does not establish what a magnitude-controlled drafter would learn end-to-end.

Table 4: Effect of pinning the hidden-state RMS to the FC’s RMS on Llama 3.1 8B during test time.

Model	Mode	Accept	Δ Acc.	Sink $k=0$	Sink $k=7$	Chain $k=7$
Pre-norm	Regular	3.06	–	0.40	0.14	0.31
Pre-norm	Normed	1.33	–56%	0.41	0.29	0.24
Post-norm	Regular	3.15	–	0.11	0.11	0.35
Post-norm	Normed	2.09	–34%	0.11	0.13	0.33

Role of Training Window. A possible additional contributor to attention drift is the training procedure itself. Most EAGLE-style trainers use a fixed context window during train-time testing (TTT), with the oldest tokens dropped from the window as drafting proceeds. This means the drafter is trained on inputs where early prompt positions, including the sink token, are progressively pushed out of context as speculation depth increases. The drafter may therefore learn to reduce reliance on these positions over the chain. While our main results identify hidden-state magnitude accumulation as a key contributor to drift, this training-window effect may further amplify sink weakening. We leave a detailed study of how training-window construction interacts with attention drift to future work.

4.5 Attention Drift in Other Architectures

Sections 4.2 and 4.3 propose two fixes for EAGLE: post-norm and gated attention. However our experiments so far cover only one model family. Here we examine whether these fixes generalize. Figure 11 shows that across drafter–target pairs, both architectural changes stabilize sink attention and self-token attention. The gated-attention models (Qwen 3.5 and the post-norm gated variant of Llama 3.1) settle at different absolute attention levels than the post-norm-only models, but they similarly do not exhibit progressive drift across the drafting chain.

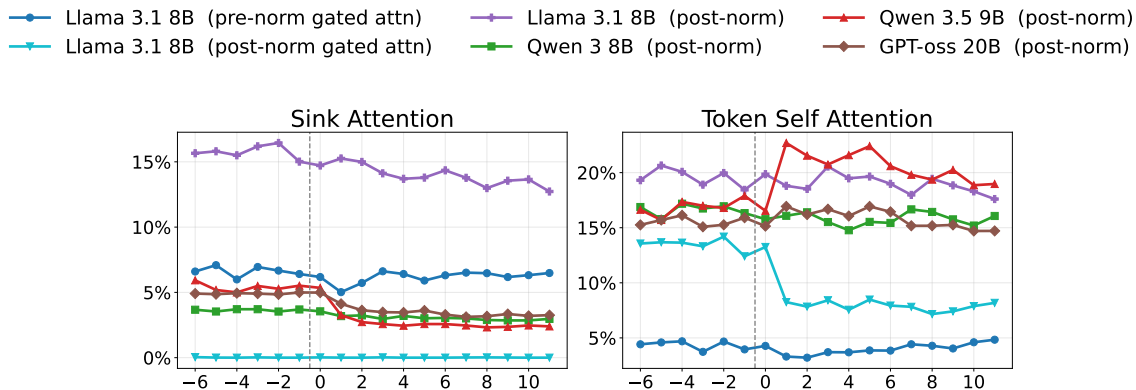


Figure 11: Attention Sink and Self-Token Attention for each model that fixes the attention drift.

MTP Heads. We observe a different drift profile in MTP heads compared to pre-norm EAGLE-3: drift onsets sharply within the first few speculation steps and then stabilizes at a new attention baseline, rather than progressing gradually across the entire chain (Figure 7). Sink attention drops sharply at speculation start, and attention to the most recently drafted token rises and then plateaus after a few steps. This is notable because MTP uses a post-norm architecture, where we expected drift to be largely suppressed. We see a similar

sharp-then-stabilize pattern in our gated-attention variants (Figure 11), suggesting that post-norm combined with attention reweighting produces a distinct dynamic from gradual pre-norm drift.

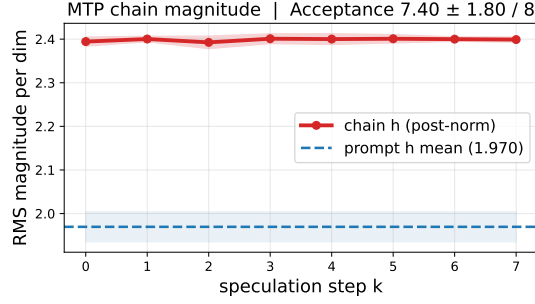


Figure 12: Magnitude drift on Qwen3.5 9B MTP heads

This is also consistent with our finding that **magnitude accumulation is one contributor to drift but not the only mechanism**. We focus our diagnosis and intervention on EAGLE-3, which trains a separate drafter with its own LM head post-hoc against the frozen target’s output distribution. MTP, in contrast, reuses the target’s LM head and is trained jointly with the target during pretraining, which may contribute to the observed effect. The exact training procedure for Qwen3.5’s MTP heads has not been publicly disclosed; characterizing the role of training loss versus architecture in the emergence of attention drift is left as future work.

5 Performance Impact

In this section, we evaluate different solutions to the attention and magnitude drift problem. We report acceptance length excluding the bonus token except for SGLang benchmarks, to compare raw performance across speculative decoding models. We trained drafters for four target models spanning different architectures: Llama 3.1 8B (dense), Qwen 3 8B (dense thinking), Qwen 3.5 9B (GDN-hybrid thinking), and GPT-OSS 20B (sparse MoE thinking).

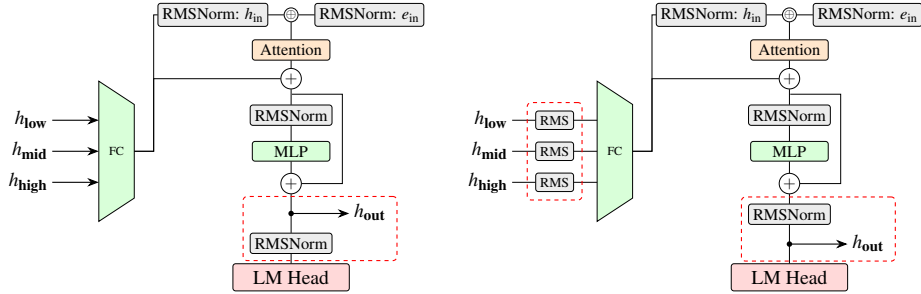


Figure 13: Standard *Pre-norm* (Left) vs proposed *Post-norm* (Right) drafter architectures.

The proposed post-norm architecture places individual RMSNorms after each target hidden state $h_{low}, h_{mid}, h_{high}$ and accumulates the drafter’s hidden states *after* the RMSNorm (Figure 13). By using post-norm, we were able to reduce the TTT length from 8 to 4, cutting training time by roughly one third without impacting performance. We also trained baseline models using the same training data and configuration with the regular pre-norm architecture for a fair comparison. **Post-norm improved acceptance length consistently across all four models, with gains ranging up to 12%** (Figure 14, full per-model results in Appendix B).

5.1 Template Sensitivity

Performance of speculative decoding models varies heavily across different deployment settings, system prompts, and chat templates. Since drafters are not trained on raw pre-training data but on processed supervised fine-tuning (SFT) conversations, we observed they heavily overfit to certain elements of the chat template.

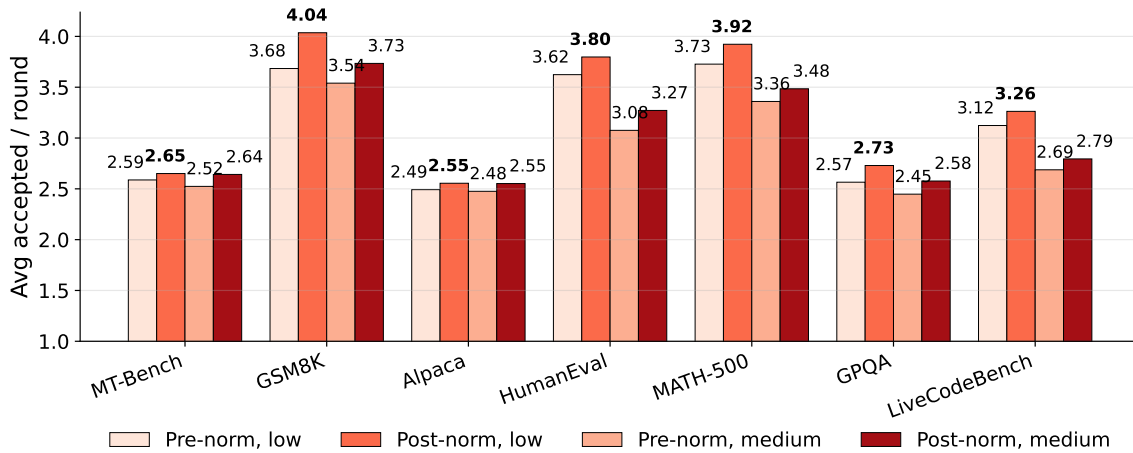


Figure 14: GPT-OSS 20B results on SGLang, acceptance length includes bonus token, temp = 0.7. Proposed post-norm shows consistent improvements over standard pre-norm.

Templates may be perturbed intentionally (e.g., disabling reasoning tags to save tokens) or unintentionally, as reported in inference engines and benchmarks. We test how well our post-norm architecture generalizes beyond the chat format and improves resilience to deployment conditions.

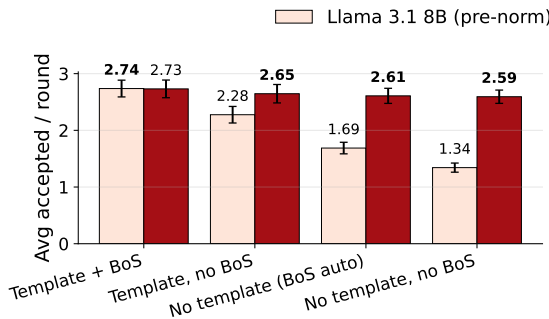


Figure 15: Template sensitivity of Llama 3.1 8B drafter pre-norm and post-norm. Temp = 0.7.

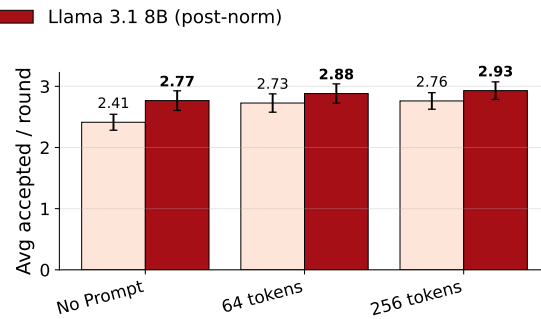


Figure 16: System prompt length's effect on accuracy for Llama 3.1 8B.

We designed two experiments to test how well our post-norm architecture generalizes beyond the training-time chat format and prompt length. In the first, we removed the chat template and sink token to measure sensitivity (Appendix D). **Post-norm was significantly more resilient to every kind of disruption**, losing at most 5% accuracy in the worst case, whereas pre-norm dropped 52% (Figure 15). Temperature set to 0.7 to prevent token-level repetition we frequently observed under template perturbation.

Is template sensitivity an artifact of attention sinks? To test this, we compared post-norm against a Gated Attention model, which eliminates the attention sink and, thus, the sensitivity to the beginning-of-sequence (BoS) token. The gated model(s) still showed high template sensitivity and in fact developed *higher* sensitivity to system prompt length, with lower overall accuracy (Figures 17 and 18). This shows that **removing the sink alone is not enough to reduce template fragility** and that fixing hidden-residual dynamics is required. We observe similar template sensitivity patterns on Qwen3-8B and GPT-oss 20B (Tables 5 and 6), where post-norm provides 5–35% gains under template perturbation, with the gap widening as templates become more disrupted.

Another factor in template sensitivity may lie in how the loss is constructed during training. In the EAGLE series, the loss is computed only on assistant tokens; user tokens don't contribute to the loss directly. So while

Table 5: **Qwen3-8B**: Avg accepted draft tokens per verification round under different prompt-template manipulations on MT-Bench.

Condition	Pre-norm	Post-norm
template + BoS	1.80 ± 0.07	1.90 ± 0.07
template, no BoS	1.76 ± 0.06	1.87 ± 0.06
no template (BoS auto)	1.71 ± 0.11	2.15 ± 0.12
no template, no BoS	1.64 ± 0.09	2.21 ± 0.10

Table 6: **GPT-oss 20B**: Avg accepted draft tokens per verification round under different prompt-template manipulations on MT-Bench.

Condition	Pre-norm	Post-norm
template + BoS	1.66 ± 0.13	1.80 ± 0.14
template, no BoS	1.55 ± 0.11	1.73 ± 0.13
no template (BoS auto)	1.43 ± 0.07	1.75 ± 0.07
no template, no BoS	1.29 ± 0.05	1.65 ± 0.07

parameters are shared and trained on the full sequence, the model’s outputs at user positions are not directly constrained, only their utility for downstream assistant-position predictions is. This asymmetry could show up especially when the chat template is changed and the boundary between supervised and unsupervised positions change from where the model was trained to expect it.

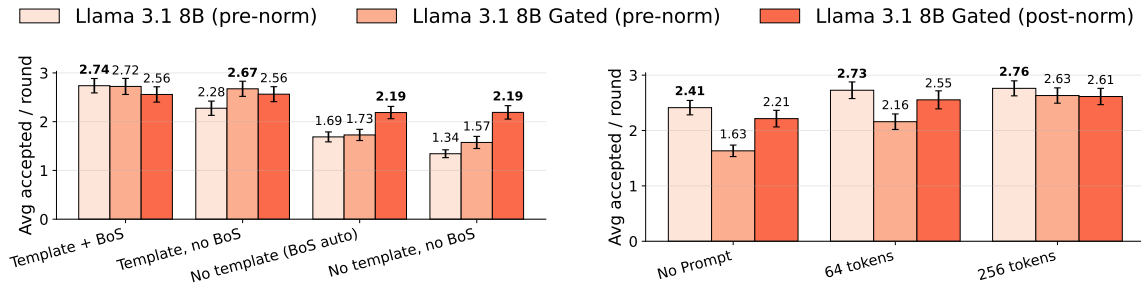


Figure 17: Gating is not sensitive to BoS, but it still overfits to other tokens in the template.

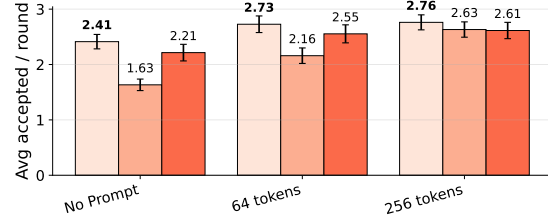


Figure 18: Gated attention overfits to the system prompt, hurting accuracy badly.

System Prompt Length. In our second experiment with temperature 0, we varied system prompt length (Llama’s default prompt, trimmed to different lengths). Post-norm beat pre-norm at every length tested (Figure 16). More importantly, **post-norm was substantially more robust to system prompt length variation**: post-norm degraded at most 5% from its peak, while pre-norm dropped 13%. Post-norm’s worst case still beats pre-norm’s best.

The cross-model differences in this sensitivity correlate with training-time prompt distribution. Llama’s default system prompt is around 500 characters, while Qwen3 and GPT-OSS are trained with much shorter system prompts (30–40 characters). This may explain why Qwen3 and GPT-OSS are more resilient to system-prompt length changes overall, and why they perform best at prompts around 64 tokens (Tables 7 and 8).

Table 7: **Qwen3-8B**: Avg accepted draft tokens per verification round as the system context grows from 0 to 256 tokens.

Condition	Pre-norm	Post-norm
no system	1.82 ± 0.06	1.94 ± 0.06
system 64 tok	1.92 ± 0.06	2.00 ± 0.07
system 256 tok	1.89 ± 0.07	1.96 ± 0.07

Table 8: **GPT-oss 20B**: Avg accepted draft tokens per verification round as the prepended system context grows from 0 to 256 tokens.

Condition	Pre-norm	Post-norm
no system	1.86 ± 0.13	2.01 ± 0.14
system 64 tok	1.94 ± 0.12	2.06 ± 0.13
system 256 tok	1.85 ± 0.12	2.02 ± 0.14

5.2 Long Context

Another challenge for speculative decoding models is long context. Drafters are relatively cheap to train and are usually trained with a short context window such as 4096 tokens, which makes them easier and more affordable to train. However, they fail catastrophically outside their trained context length. LLMs have developed various techniques to handle this, one being sliding window attention (SWA), where the model’s

effective context is a fixed window over the most recent tokens. This makes intuitive sense for drafters in particular: they are weaker by design, and their predictions on easy tokens shouldn’t depend on long-range context. SWA has been studied for long-context speculative decoding from both serving-systems [15] and drafter-training angles [4]. We use SWA as a long-context evaluation tool rather than as a method we propose; our post-norm fix is orthogonal to these works and can be combined with existing long-context speculative decoding techniques.

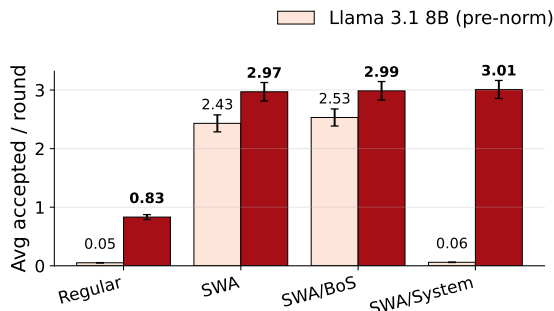


Figure 19: Accuracy of Llama 3.1 8B drafter on various long-context multi-turn chat settings.

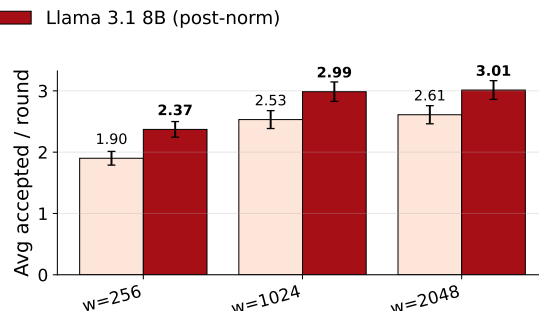


Figure 20: Effect of window size on prediction accuracy.

We created a multi-turn conversation benchmark with context length beyond our models’ trained length. A multi-turn setup isolates the model’s long-context capability from degradation caused by unrelated filler text (results summarized in Figure 21). Under **full attention** (regular), the pre-norm model fails catastrophically, dropping to 0.05 average acceptance length, while post-norm drops to 0.83, also unusable, but $15\times$ better. With **SWA**, most of the accuracy is rescued: pre-norm recovers 88% of its single-turn baseline, while post-norm matches and slightly exceeds its single-turn performance. Since pre-norm develops a strong sink, **carrying the BoS token** (SWA/BoS) through the window helps it more (up to 91% recovery), with only a minor bump for post-norm. Following on this, we hypothesized that **carrying the system prompt** (SWA/System) would add additional context. For post-norm this added another 1%, but pre-norm collapsed to near-zero, it could not accommodate the wider range of positional embeddings introduced by the longer prefix. **Across all SWA conditions, post-norm outperformed pre-norm by 20%.**

True long-context performance. In addition to our multi-turn benchmark, we evaluate on *LongBench* [16] across three task categories: summarization, few-shot learning, and coding. Absolute task scores are lower than in the multi-turn setting due to the inherent difficulty of long-context understanding. **Post-norm outperformed pre-norm by 20–25% across all categories and context lengths (Table 9).** On the government report summarization task, adding the system prompt yielded an additional 8% gain, suggesting that informative prefixes can further help SWA-based drafters recover context, though we leave a broader study of this effect to future work.

Table 9: LongBench: avg accepted draft tokens per round, per task and SWA mode. Window=1024.

Task	SWA + BoS		SWA + Sys Prompt	
	Pre-norm	Post-norm	Pre-norm	Post-norm
Gov Report (Summarization)	1.35	1.57	0.47	1.70
Samsum (Few-Shot)	1.04	1.31	0.39	1.31
Repobench (Coding)	1.85	2.26	0.26	2.25

How much does the drafter rely on long-range context vs. recent tokens? To answer this, we varied the SWA window size (Figure 20). Across all three window sizes tested, post-norm maintained a consistent



Figure 21: Attended tokens (marked in orange) in different SWA implementations.

gap over pre-norm. Even a small 256-token window recovers 80% of the full-context baseline; performance saturates around 1024 tokens, with diminishing returns beyond that. This suggests **the model mostly cares about recent tokens and SWA can substantially reduce compute without meaningfully hurting drafter accuracy.**

6 Related Work

Speculative decoding accelerates LLM inference by drafting candidate tokens and verifying them in parallel [1]. Several drafter designs have been proposed: Medusa [17] attaches multiple prediction heads to the target, Hydra [18] introduces sequential dependence between these heads, and D-Flash [19] uses diffusion models to predict sequences in one forward pass. We focus on EAGLE-3 as the dominant auto-regressive drafter design.

Attention sinks and patterns. Xiao et al. [12] was the first to observe attention sinks and show that preserving these sinks is essential for long-context perplexity under sliding-window attention. Qiu et al. [20] proposed gated attention as a mechanism to reduce the model’s reliance on a fixed sink token. Both works study sink behavior in the target model itself; we are the first, to our knowledge, to characterize attention behavior in speculative decoding drafters and identify a distinct failure mode (attention drift) specific to the chain-residual structure of these models.

Normalization in transformers. The placement of normalization relative to residual connections has been studied extensively in the standard transformer setting. Xiong et al. [21], Wu et al. [22] showed that pre-norm enables more stable training than post-norm, leading most modern LLMs to adopt pre-norm. Our work revisits this question for speculative decoding drafters, where iterative self-feedback across speculation steps changes the dynamics. We show that post-norm becomes necessary to prevent magnitude accumulation that drives attention drift.

7 Limitations

Our study has two main limitations. First, we focus on EAGLE-3 as the dominant post-training auto-regressive drafter in production. We observe attention drift in MTP heads, suggesting the phenomenon extends beyond EAGLE-3, but we do not investigate the alternative mechanisms at play there or evaluate our fix across other drafter designs. Extending the analysis to MTP, Hydra, and other variants is a natural direction for future work. Second, our experiments are limited to models below 120B parameters due to compute constraints; behavior at larger scale remains unverified.

8 Conclusion

We identified *attention drift* in auto-regressive speculative decoding drafters: as the drafter generates successive tokens during speculation, attention progressively migrates from the sink onto recently-generated tokens. We characterized this phenomenon across model families and showed that it is closely tied to monotonic magnitude growth in the unnormalized residual path between speculation steps, which causes the drafter to behave more like additional transformer layers stacked on the target than as a standalone auto-regressive predictor. Two simple architectural interventions, post-norm on the drafter output and RMSNorm before the target hidden projection, address the underlying dynamics and yield consistent improvements, with the largest gains under deployment shift. Crucially, our experiments show that fixing drift alone is not enough to improve performance; **attention drift is merely a symptom of a deeper issue, not its cause.** Post-norm helps because it addresses the underlying dynamics: it prevents magnitude accumulation and stops the drafter from learning depth-specific transformations, which together improve both performance and robustness. We hope that naming attention drift gives the community a useful diagnostic for drafter robustness, and that post-norm proves a useful default for production systems.

Acknowledgments and Disclosure of Funding

We thank fal and Lambda for the compute grants that supported this research.

References

- [1] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pages 19274–19286. PMLR, 2023.
- [2] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.
- [3] Guangxuan Xiao, Jiaming Tang, Jingwei Zuo, junxian guo, Shang Yang, Haotian Tang, Yao Fu, and Song Han. Duoattention: Efficient long-context LLM inference with retrieval and streaming heads. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=cFu7ze7xUm>.
- [4] Penghui Yang, Cunxiao Du, Fengzhuo Zhang, Haonan Wang, Tianyu Pang, Chao Du, and Bo An. Longspec: Long-context lossless speculative decoding with efficient drafting and verification. In *ES-FoMo III: 3rd Workshop on Efficient Systems for Foundation Models*, 2025. URL <https://openreview.net/forum?id=GFN9PWbfHs>.
- [5] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=4exx1hUffq>.
- [6] Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction. In Ruslan Salakhutdinov, Zico Kolter, Katherine A. Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*, Proceedings of Machine Learning Research, pages 15706–15734. PMLR / OpenReview.net, 2024. URL <https://proceedings.mlr.press/v235/gloeckle24a.html>.
- [7] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: speculative sampling requires rethinking feature uncertainty. In Ruslan Salakhutdinov, Zico Kolter, Katherine A. Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp, editors, *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*, Proceedings of Machine Learning Research, pages 28935–28948. PMLR / OpenReview.net, 2024. URL <https://proceedings.mlr.press/v235/li24bt.html>.
- [8] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle-2: Faster inference of language models with dynamic draft trees. In *Proceedings of the 2024 conference on empirical methods in natural language processing*, pages 7421–7432, 2024.
- [9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [10] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626, 2023.
- [11] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody H Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E Gonzalez, et al. Sglang: Efficient execution of structured language model programs. *Advances in neural information processing systems*, 37:62557–62583, 2024.
- [12] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=NG7sS51zVF>.

- [13] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in neural information processing systems*, 36:46595–46623, 2023.
- [14] Kimi Team, Guangyu Chen, Yu Zhang, Jianlin Su, Weixin Xu, Siyuan Pan, Yaoyu Wang, Yucheng Wang, Guanduo Chen, Bohong Yin, et al. Attention residuals. *arXiv preprint arXiv:2603.15031*, 2026.
- [15] Ranajoy Sadhukhan, Jian Chen, Zhuoming Chen, Vashisth Tiwari, Ruihang Lai, Jinyuan Shi, Ian En-Hsu Yen, Avner May, Tianqi Chen, and Beidi Chen. Magicdec: Breaking the latency-throughput tradeoff for long context generation with speculative decoding. In *ICLR*, 2025. URL <https://openreview.net/forum?id=CS2JWaziYr>.
- [16] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, et al. Longbench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*, pages 3119–3137, 2024.
- [17] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *Forty-first International Conference on Machine Learning*, 2024. URL <https://openreview.net/forum?id=PEpbUobfJv>.
- [18] Zachary Ankner, Rishab Parthasarathy, Aniruddha Nrusimha, Christopher Rinard, Jonathan Ragan-Kelley, and William Brandon. Hydra: Sequentially-dependent draft heads for medusa decoding. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=FbhjirzvJG>.
- [19] Jian Chen, Yesheng Liang, and Zhijian Liu. Dflash: Block diffusion for flash speculative decoding. *arXiv preprint arXiv:2602.06036*, 2026.
- [20] Zihan Qiu, Zekun Wang, Bo Zheng, Zeyu Huang, Kaiyue Wen, Songlin Yang, Rui Men, Le Yu, Fei Huang, Suozhi Huang, Dayiheng Liu, Jingren Zhou, and Junyang Lin. Gated attention for large language models: Non-linearity, sparsity, and attention-sink-free. In D. Belgrave, C. Zhang, H. Lin, R. Pascanu, P. Koniusz, M. Ghassemi, and N. Chen, editors, *Advances in Neural Information Processing Systems*, volume 38, pages 100092–100118. Curran Associates, Inc., 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/file/904e89bb4e632e75fb47f093b620b257-Paper-Conference.pdf.
- [21] Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tieyan Liu. On layer normalization in the transformer architecture. In *International conference on machine learning*, pages 10524–10533. PMLR, 2020.
- [22] Xinyi Wu, Amir Ajorlou, Yifei Wang, Stefanie Jegelka, and Ali Jadbabaie. On the role of attention masks and layernorm in transformers. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=1IH6oCdppg>.

A Gated attention

In the gated attention variant of our models, we add an optional per-element sigmoid gate over the hidden dimension to the EAGLE3 draft self-attention. Let $\mathbf{x} \in \mathbb{R}^{T \times d_{\text{in}}}$ denote the input stream to the attention block. Because EAGLE3 fuses the verifier hidden state with the input-embedding inside the decoder layer, $\mathbf{x}$ is the concatenation of the two and $d_{\text{in}} = 2d$, where d is the draft hidden size. The query, key and value projections operate on $\mathbf{x}$ as in standard Llama, and we write the multi-head attention output *before* the output projection as

$$\mathbf{A} = \text{Attn}(\mathbf{x}\mathbf{W}_Q, \mathbf{x}\mathbf{W}_K, \mathbf{x}\mathbf{W}_V) \in \mathbb{R}^{T \times h d_h},$$

with h heads of dimension d_h . Our modification introduces a learned gate $\mathbf{W}_g \in \mathbb{R}^{d_{\text{in}} \times h d_h}$ that shares its input with the QKV projections,

$$\mathbf{g} = \sigma(\mathbf{x}\mathbf{W}_g), \quad (1)$$

where $\sigma(\cdot)$ denotes the element-wise sigmoid ($\sigma(z) = \frac{1}{1 + e^{-z}}$). The gate is applied to the concatenated head output and *precedes* the output projection $\mathbf{W}_O$:

$$\mathbf{y} = (\mathbf{g} \odot \mathbf{A}) \mathbf{W}_O. \quad (2)$$

B Benchmark Results

We report SGLang acceptance length (including the bonus token) per benchmark for each target model trained in this work. Tables 6–9 cover GPT-oss 20B, Qwen3.5 9B, Qwen3 8B, and Llama 3.1 8B respectively, sweeping the relevant decoding modes for each (low/medium reasoning effort for GPT-oss; thinking/no-thinking for Qwen3 and Qwen3.5). Across all four models and every decoding mode, post-norm matches or improves on pre-norm; the single regression is Llama 3.1 8B on MATH-500, which falls within evaluation noise. Gains are largest on math and code (e.g., GSM8K +10% on GPT-oss low, HumanEval +5% on Qwen3 8B no-think) and smallest on Alpaca-style open-ended chat, consistent with the harder, longer-horizon completions benefiting more from a stable drafter.

Table 10: SGLang acceptance length for **GPT-oss 20B** + EAGLE3 across pre/post-norm draft and low/medium reasoning effort. Best per row in bold.

Benchmark	Pre, low	Post, low	Pre, medium	Post, medium
MT-Bench	2.59	2.65	2.52	2.64
GSM8K	3.68	4.04	3.54	3.73
Alpaca	2.49	2.55	2.48	2.55
HumanEval	3.62	3.80	3.08	3.27
MATH-500	3.73	3.92	3.36	3.48
GPQA	2.57	2.73	2.45	2.58
LiveCodeBench	3.12	3.26	2.69	2.79

Table 11: SGLang acceptance length for **Qwen3.5 9B** + EAGLE3 across pre/post-norm draft and thinking/no-thinking decoding. Best per row in bold.

Benchmark	Pre, no-think	Post, no-think	Pre, think	Post, think
MT-Bench	3.08	3.16	3.60	3.69
GSM8K	4.21	4.28	4.32	4.42
Alpaca	2.73	2.79	3.49	3.57
HumanEval	4.47	4.49	4.43	4.43
MATH-500	5.08	5.12	4.95	5.01
GPQA	3.54	3.65	3.78	3.87
LiveCodeBench	4.31	4.36	4.22	4.30

Table 12: SGLang acceptance length for **Qwen3 8B** + EAGLE3 across pre/post-norm draft and thinking/no-thinking decoding. Best per row in bold.

Benchmark	Pre, no-think	Post, no-think	Pre, think	Post, think
MT-Bench	3.55	3.61	3.09	3.15
GSM8K	5.58	5.73	3.73	3.83
Alpaca	3.14	3.35	2.68	2.82
HumanEval	5.00	5.21	3.05	3.13
MATH-500	5.80	5.89	3.57	3.64
GPQA	3.91	4.04	2.98	3.08
LiveCodeBench	4.59	4.71	3.06	3.14

Table 13: SGLang acceptance length per benchmark for **Llama 3.1 8B** with EAGLE3 (steps=7, topk=1, draft=8). Best per row in bold. The single regression on MATH-500 is within evaluation noise.

Benchmark	Pre-norm	Post-norm
MT-Bench	3.84	3.90
GSM8K	5.28	5.33
Alpaca	3.52	3.59
HumanEval	5.56	5.59
MATH-500	5.87	5.81
GPQA	4.74	4.78
LiveCodeBench	4.16	4.22

B.1 Model Long-Context SWA Sensitivity

To our surprise, the Gated Post-Norm model did not suffer at all from out-of-distribution inference lengths and showed no accuracy loss in the full-context attention case. This could be attributed to its low attention entropy: it learns to attend to only a couple of tokens, and this sharpened attention pattern may filter the destructive signal via the sigmoid gate. This also suggests the model relies less on absolute positional information than the baseline does.

Table 14: **Llama 3.1 8B**: Avg accepted draft tokens per round at 8k context, comparing full attention to sliding-window variants with optional BoS / system-prompt carry.

Condition	pre-norm	post-norm	Gated pre-norm	Gated post-norm
full attn	0.05 $\pm$ 0.00	0.83 $\pm$ 0.04	0.84 $\pm$ 0.06	2.74 $\pm$ 0.17
SWA	2.43 $\pm$ 0.14	2.97 $\pm$ 0.16	1.21 $\pm$ 0.09	2.70 $\pm$ 0.17
SWA + BoS	2.53 $\pm$ 0.15	2.99 $\pm$ 0.16	1.22 $\pm$ 0.09	2.69 $\pm$ 0.17
SWA + system	0.06 $\pm$ 0.00	3.01 $\pm$ 0.15	1.72 $\pm$ 0.12	2.70 $\pm$ 0.17

Table 15: **GPT-oss 20b**: Avg accepted draft tokens per round at 16k context, comparing full attention to sliding-window variants.

Condition	Pre-norm	Post-norm
full attn	1.67 $\pm$ 0.09	1.67 $\pm$ 0.09
SWA	1.09 $\pm$ 0.08	1.94 $\pm$ 0.11

Table 16: **Qwen3-8B**: Avg accepted draft tokens per round at 8k context, comparing full attention to sliding-window variants with optional BoS / system-prompt carry.

Condition	Pre-norm	Post-norm
full attn	1.33 $\pm$ 0.04	1.24 $\pm$ 0.04
SWA	1.87 $\pm$ 0.07	2.05 $\pm$ 0.07
SWA + BoS	1.97 $\pm$ 0.07	2.08 $\pm$ 0.07
SWA + system	1.87 $\pm$ 0.07	2.05 $\pm$ 0.07

B.2 SWA Size / Accuracy

Section 5.2 reports the window-size sweep for Llama 3.1 8B at 8k context. Table 20 confirms the same pattern holds for Qwen3 8B at a 32k target context: post-norm leads pre-norm at every window size, and gains saturate around $w=1024$ with only marginal improvement at $w=2048$.

Table 17: **Llama-3.1 8B**: Avg accepted draft tokens per round under SWA + BoS carry as the window size grows from 256 to 2048 (target context 8192).

Condition	Pre-norm	Post-norm
w=256	1.90 $\pm$ 0.11	2.37 $\pm$ 0.13
w=1024	2.53 $\pm$ 0.15	2.99 $\pm$ 0.16
w=2048	2.61 $\pm$ 0.15	3.01 $\pm$ 0.15

Table 18: **Qwen3-8B**: Avg accepted draft tokens per round under SWA + BoS carry as the window size grows from 256 to 2048 (target context 32768).

Condition	Pre-norm	Post-norm
w=256	1.64 $\pm$ 0.08	1.82 $\pm$ 0.08
w=1024	1.97 $\pm$ 0.09	2.14 $\pm$ 0.09
w=2048	2.00 $\pm$ 0.09	2.16 $\pm$ 0.09

B.3 LongBench Results

We further break down LongBench-E results for Llama 3.1 8B by prompt-length bucket, using each architecture’s preferred SWA mode (SWA+BoS for pre-norm, SWA+System Prompt for post-norm. Post-norm outperforms pre-norm in every bucket from 0–4k all the way to 32–36k. This indicates that the post-norm advantage seen on the multi-turn benchmark in Section 5.2 transfers to true long-context tasks rather than being specific to the multi-turn setting.

Table 19: **Llama 3.1 8B, LongBench-E**: avg accepted draft tokens per round vs prompt length, averaged over all tasks. Each model uses its preferred SWA mode (pre-norm: SWA + BoS, post-norm: SWA + Sys Prompt). n is the number of verification rounds in each bucket.

Ctx (tokens)	Pre-norm (SWA + BoS)		Post-norm (SWA + Sys Prompt)	
	Acceptance Length (τ)	Number of samples	Acceptance Length (τ)	n
0-4k	1.60	7838	1.99	6802
4-8k	1.63	11146	2.07	9563
8-12k	1.30	9927	1.73	8354
12-16k	1.28	6242	1.60	5482
16-20k	1.82	3000	1.98	2839
20-24k	1.94	3855	1.96	3831
24-28k	1.77	2744	2.20	2374
28-32k	1.73	1560	2.25	1309
32-36k	1.21	542	1.80	428

C Training

We have used a modified instance of the *SpecForge* repository to train our models. This repository implements the Train-Time-Test method for EAGLE-3 training and it is the framework used to train current state-of-the-art EAGLE-3 models.

Llama and Qwen3 variants are trained on the Open-PerfectBlend dataset with regenerated answers using target model. The dataset consists over 1.4M samples. We have run it for 2 epochs and 1.5×10^{-4} LR, with effective batch size of 4. On average training took around 48 H200 hours for each model with 8-9B parameters.

Qwen3.5 and Gpt-oss variants were trained on Nemotron post training dataset with regenerated answers using target model. The dataset consist over 1.4M samples. The maximum sequence length was 8K tokens. We have run it for 1 epoch and 1.5×10^{-4} LR. On average training took around 36 to 48 H200 hours based on model size (20B and 120B).

D Template Perturbations

--- regular ---

<|begin_of_text|><|start_header_id|>system<|end_header_id|>

You are a helpful assistant.<|eot_id|><|start_header_id|>user<|end_header_id|>

What is the capital of France?<|eot_id|><|start_header_id|>assistant<|end_header_id|>

--- no_bos ---

<|start_header_id|>system<|end_header_id|>

You are a helpful assistant.<|eot_id|>...

--- no_template ---

<|begin_of_text|>Question: What is the capital of France?

Answer:

--- no_bos_no_template ---

Question: What is the capital of France?

Answer: